EXPERIMENT - 4

AIM: Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, selfjoin, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

SOURCE CODE:

```
CREATE TABLE Department ( dept_id
   INT PRIMARY KEY, dept_name
   VARCHAR(50)
);
CREATE TABLE Project ( project_id
   INT PRIMARY KEY, project_name
   VARCHAR(50), dept_id INT,
   FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Employee ( emp_id
   INT PRIMARY KEY, emp_name
   VARCHAR(50), dept_id INT,
   project_id INT,
   salary DECIMAL(10,2), city
   VARCHAR(50),
   FOREIGN KEY (dept_id) REFERENCES Department(dept_id), FOREIGN KEY
   (project_id) REFERENCES Project(project_id)
);

INSERT INTO Department VALUES
(1, 'HR'), (2, 'Finance'), (3, 'IT'), (4, 'Marketing'), (5, 'Operations');

INSERT INTO Project VALUES
(101, 'Recruitment System', 1), (102, 'Payroll Automation', 2),
(103, 'Cybersecurity Upgrade', 3), (104, 'Website Revamp', 3),
(105, 'Ad Campaign', 4), (106, 'Market Research', 4),
(107, 'Logistics Optimization', 5), (108, 'Inventory System', 5);

INSERT INTO Employee VALUES
(1, 'Alice', 1, 101, 45000, 'Delhi'),
(2, 'Bob', 1, 101, 42000, 'Mumbai'),
(3, 'Charlie', 1, 101, 47000, 'Pune'),
(4, 'David', 2, 102, 55000, 'Delhi'),
(5, 'Eva', 2, 102, 60000, 'Lucknow'),
(6, 'Frank', 2, 102, 52000, 'Chennai'),
(7, 'Grace', 3, 103, 70000, 'Bangalore'),
(8, 'Hannah', 3, 103, 68000, 'Hyderabad'),
(9, 'Ian', 3, 104, 72000, 'Delhi'),
```

```
(10, 'Jack', 3, 104, 65000, 'Pune'),
(11, 'Karen', 3, 104, 64000, 'Mumbai'),
(12, 'Leo', 4, 105, 48000, 'Delhi'),
(13, 'Mia', 4, 105, 50000, 'Lucknow'),
(14, 'Nina', 4, 106, 51000, 'Chennai'),
(15, 'Oscar', 4, 106, 53000, 'Delhi'),
(16, 'Paul', 5, 107, 56000, 'Hyderabad'),
(17, 'Quinn', 5, 107, 57000, 'Bangalore'),
(18, 'Rita', 5, 107, 59000, 'Delhi'),
(19, 'Sam', 5, 108, 60000, 'Lucknow'),
(20, 'Tina', 5, 108, 61000, 'Mumbai'),
(21, 'Uma', 5, 108, 62000, 'Chennai'),
(22, 'Victor', 2, 102, 53000, 'Delhi'),
(23, 'Wendy', 2, 102, 54000, 'Hyderabad'),
(24, 'Xavier', 3, 103, 71000, 'Bangalore'),
(25, 'Yara', 3, 104, 66000, 'Lucknow'),
(26, 'Zane', 4, 105, 49000, 'Delhi'),
(27, 'Aditi', 4, 106, 52000, 'Mumbai'),
(28, 'Bhavya', 1, 101, 46000, 'Chennai'),
(29, 'Chetan', 1, 101, 43000, 'Lucknow'),
(30, 'Devika', 5, 107, 58000, 'Delhi');

SELECT E.emp_id, E.emp_name, D.dept_name FROM
Employee E
INNER JOIN Department D ON
   E.dept_id = D.dept_id;

SELECT D.dept_id, D.dept_name, E.emp_name FROM
Department D
LEFT JOIN Employee E
   ON D.dept_id = E.dept_id;

SELECT E1.emp_name AS Employee1, E2.emp_name AS Employee2, E1.dept_id FROM
Employee E1
INNER JOIN Employee E2  ON
   E1.dept_id = E2.dept_id
   AND E1.emp_id < E2.emp_id;

SELECT E.emp_name, P.project_name, D.dept_name
FROM Employee E
INNER JOIN Project P
   ON E.project_id = P.project_id
INNER JOIN Department D
   ON E.dept_id = D.dept_id;
```

```
SELECT E.emp_name, E.dept_id, E.salary FROM
Employee E
WHERE E.salary > ( SELECT
   AVG(E2.salary) FROM
   Employee E2
   WHERE E2.dept_id = E.dept_id );

SELECT E.emp_id, E.emp_name, E.dept_id FROM
Employee E
WHERE EXISTS ( SELECT 1
   FROM Project P WHERE P.dept_id = E.dept_id );

SELECT emp_name FROM
Employee WHERE emp_id
IN (
   SELECT emp_id FROM
   Employe WHERE city =
   'Delhi' )
AND emp_id IN ( SELECT
   emp_id FROM
   Employee
   WHERE salary > 60000 );

SELECT emp_name FROM
Employee
WHERE emp_id NOT IN (
   SELECT emp_id   FROM
   Employee
   WHERE city = 'Delhi' );

EXPLAIN
SELECT E.emp_name, D.dept_name FROM
Employee E
INNER JOIN Department D ON
   E.dept_id = D.dept_id;

EXPLAIN
SELECT D.dept_name, E.emp_name FROM
Department D
LEFT JOIN Employee E
   ON D.dept_id = E.dept_id;

EXPLAIN
SELECT E.emp_name, E.salary FROM
Employee E
WHERE E.salary > ( SELECT
```

```
    AVG(E2.salary) FROM
    Employee E2
    WHERE E2.dept_id = E.dept_id );

EXPLAIN
SELECT E.emp_name FROM Employee E WHERE
EXISTS (
    SELECT 1
    FROM Project P WHERE P.dept_id = E.dept_id );
```

OUTPUT :



EXPERIMENT - 5

AIM: To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

SOURCE CODE:

```
CREATE TABLE IF NOT EXISTS Department(
Dept_ID INT PRIMARY KEY,
Dept_Name VARCHAR(100) NOT NULL
);
CREATE TABLE IF NOT EXISTS Employee(
Emp_ID INT PRIMARY KEY,
Emp_Name VARCHAR(100) NOT NULL,
Job_Title VARCHAR(100),
Salary DECIMAL(10,2),
Dept_ID INT,
FOREIGN KEY(Dept_ID) REFERENCES Department(Dept_ID)
);

INSERT IGNORE INTO Department(Dept_ID,Dept_Name) VALUES
(1,'Engineering'),
(2,'Sales'),
(3,'Human Resources'),
(4,'Finance'),
(5,'Operations');

INSERT IGNORE INTO Employee(Emp_ID,Emp_Name,Job_Title,Salary,Dept_ID) VALUES
(1,'Ava Patel','Developer',60000,1),
(2,'Liam Chen','Developer',62000,1),
(3,'Mia Garcia','Analyst',58000,1),
(4,'Noah Smith','Engineering Manager',85000,1),
(5,'Olivia Jones','Developer',61000,1),
(6,'Ethan Brown','Analyst',56000,1),
(7,'Sophia Wilson','Sales Representative',52000,2),
(8,'Lucas Davis','Sales Representative',54000,2),
(6,'Amelia Miller','Sales Manager',78000,2),
(10,'James Taylor','Sales Representative',51000,2),
(11,'Isabella Anderson','Sales Representative',53000,2),
(12,'Benjamin Thomas','Sales Representative',55000,2),
(13,'Charlotte Moore','HR Specialist',57000,3),
(14,'Elijah Martin','HR Specialist',56000,3),
(15,'Harper Lee','Recruiter',56000,3),
(16,'Daniel White','HR Manager',80000,3),
(17,'Evelyn Harris','HR Specialist',58000,3),
(18,'Henry Clark','Recruiter',60000,3),
(16,'Abigail Lewis','Accountant',63000,4),
(20,'Sebastian Walker','Accountant',64000,4),
(21,'Emily Hall','Finance Manager',88000,4),
```

```
(22,'Jack Allen','Accountant',65000,4),
(23,'Ella Young','Financial Analyst',67000,4),
(24,'Owen King','Financial Analyst',66000,4),
(25,'Grace Wright','Operations Specialist',55000,5),
(26,'Matthew Scott','Operations Specialist',56000,5),
(27,'Chloe Green','Operations Manager',82000,5),
(28,'Wyatt Baker','Operations Specialist',57000,5),
(26,'Lily Adams','Coordinator',52000,5),
(30,'Gabriel Nelson','Coordinator',54000,5);

CREATE VIEW Department_Salary_Summary AS
SELECT D.Dept_ID,D.Dept_Name,
COUNT(E.Emp_ID) AS Total_Employees,
AVG(E.Salary) AS Average_Salary,
MAX(E.Salary) AS Maximum_Salary,
MIN(E.Salary) AS Minimum_Salary
FROM Department D
LEFT JOIN Employee E ON D.Dept_ID=E.Dept_ID
GROUP BY D.Dept_ID,D.Dept_Name;

SELECT * FROM Department_Salary_Summary;

ALTER TABLE Employee ADD Manager_ID INT NULL;

ALTER TABLE Employee
ADD CONSTRAINT FK_Employee_Manager
FOREIGN KEY(Manager_ID) REFERENCES Employee(Emp_ID);

UPDATE Employee
SET Manager_ID=4
WHERE Emp_ID IN(1,2,3,5,6);

UPDATE Employee
SET Manager_ID=6
WHERE Emp_ID IN(7,8,10,11,12);

UPDATE Employee
SET Manager_ID=16
WHERE Emp_ID IN(13,14,15,17,18);

UPDATE Employee
SET Manager_ID=21
WHERE Emp_ID IN(16,20,22,23,24);

UPDATE Employee
SET Manager_ID=27
WHERE Emp_ID IN(25,26,28,26,30);

CREATE VIEW Employee_Hierarchy AS
SELECT E.Emp_ID,E.Emp_Name,E.Job_Title,E.Salary,E.Manager_ID,
M.Emp_Name AS Manager_Name

FROM Employee E
```

```
LEFT JOIN Employee M ON E.Manager_ID=M.Emp_ID;

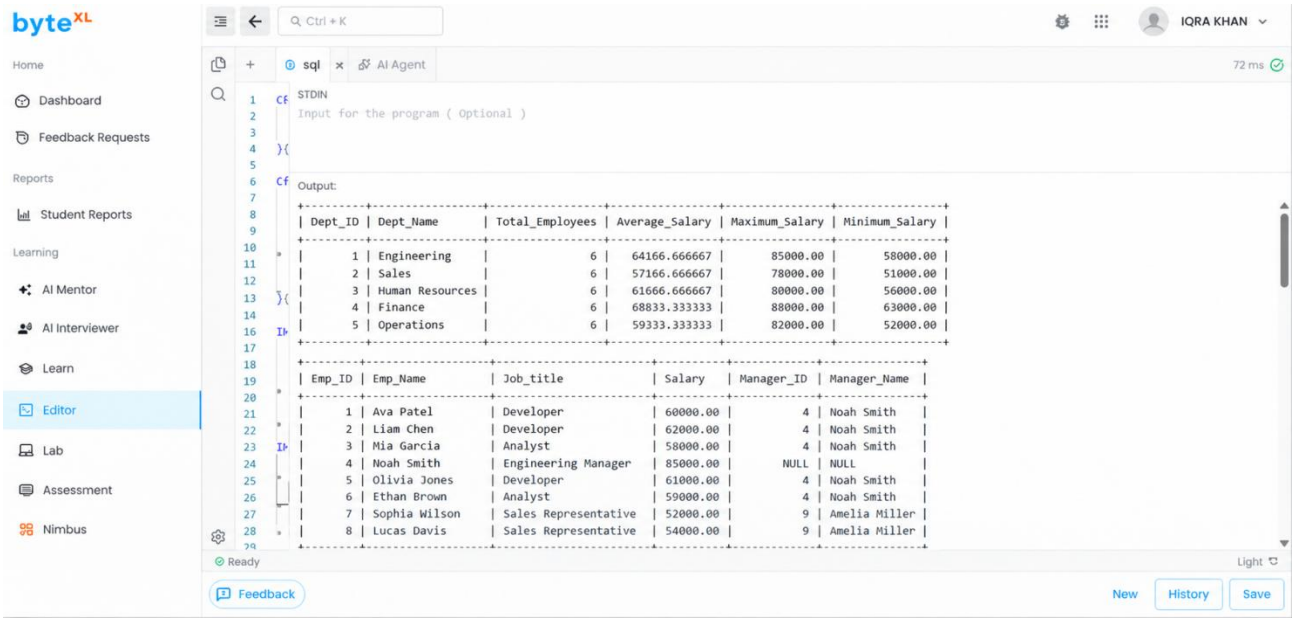
SELECT * FROM Employee_Hierarchy;

UPDATE Employee
SET Salary=60000
WHERE Emp_ID=1;

SELECT Emp_ID,Emp_Name,Salary
FROM Employee
WHERE Emp_ID=1;

WITH RECURSIVE Employee_Chain AS(
SELECT Emp_ID,Emp_Name,Manager_ID,1 AS Level
FROM Employee
WHERE Manager_ID IS NULL
UNION ALL
SELECT E.Emp_ID,E.Emp_Name,E.Manager_ID,EC.Level+1
FROM Employee E
INNER JOIN Employee_Chain EC ON E.Manager_ID=EC.Emp_ID
)
SELECT Emp_ID,Emp_Name,Manager_ID,Level
FROM Employee_Chain
    ORDER BY Level,Emp_ID;
```

OUTPUT :



EXPERIMENT - 6

AIM: To create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling, and to implement triggers for salary validation and audit logging, including testing of edge cases.

SOURCE CODE:

```
CREATE TABLE Department (
   Dept_ID INT PRIMARY KEY,
   Dept_Name VARCHAR(100) NOT NULL
);
CREATE TABLE Employee (
   Emp_ID INT PRIMARY KEY,
   Emp_Name VARCHAR(100) NOT NULL,
   Salary DECIMAL(10,2) NOT NULL,
   Dept_ID INT,
   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

INSERT INTO Department (Dept_ID, Dept_Name)
VALUES (1, 'Sales'), (2, 'Engineering');

INSERT INTO Employee (Emp_ID, Emp_Name, Salary, Dept_ID)
VALUES (1, 'Alex', 60000.00, 1);

CREATE TABLE Salary_Audit (
   Audit_ID INT AUTO_INCREMENT PRIMARY KEY,
   Emp_ID INT,
   Old_Salary DECIMAL(10,2),
   New_Salary DECIMAL(10,2),
   Changed_At TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER $$

CREATE PROCEDURE transfer_employee(
   IN p_emp_id INT,
   IN p_new_dept_id INT
)
BEGIN
   DECLARE emp_count INT DEFAULT 0;
   DECLARE dept_count INT DEFAULT 0;

   SELECT COUNT(*) INTO emp_count
   FROM Employee
   WHERE Emp_ID = p_emp_id;

   SELECT COUNT(*) INTO dept_count
   FROM Department
   WHERE Dept_ID = p_new_dept_id;
```

```
 IF emp_count = 0 THEN
      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'Employee does not exist';
   ELSEIF dept_count = 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Department does not exist';
   ELSE
      UPDATE Employee
      SET Dept_ID = p_new_dept_id
      WHERE Emp_ID = p_emp_id;
   END IF;
END $$

DELIMITER ;

CALL transfer_employee(1, 2);

SELECT *
FROM Employee
WHERE Emp_ID = 1;

-- CALL transfer_employee(666, 2);
-- CALL transfer_employee(1, 666);

DELIMITER $$

CREATE TRIGGER before_employee_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
   IF NEW.Salary < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Salary cannot be negative';
   END IF;
END $$

DELIMITER ;

-- UPDATE Employee SET Salary = -5000 WHERE Emp_ID = 1;

DELIMITER $$

CREATE TRIGGER after_employee_salary_update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
   IF OLD.Salary <> NEW.Salary THEN
      INSERT INTO Salary_Audit (
         Emp_ID,
         Old_Salary,
         New_Salary
      )
```

```
        VALUES (

            OLD.Emp_ID,
            OLD.Salary,

            NEW.Salary
        );
    END IF;
END $$

DELIMITER ;

UPDATE Employee
SET Salary = 65000
WHERE Emp_ID = 1;

SELECT *
FROM Salary_Audit;

SELECT Emp_ID, Emp_Name, Salary
FROM Employee
WHERE Emp_ID = 1;
```

OUTPUT :



EXPERIMENT - 7

AIM: To normalize a denormalized Sales table from UNF to 3NF and BCNF, and to verify lossless join and dependency preservation.

SOURCE CODE:

```
CREATE TABLE Sales_UNF (
Sale_ID INT, Customer_ID INT,
Customer_Name VARCHAR(50), Customer_City VARCHAR(50), Product_ID INT,
Product_Name VARCHAR(50), Quantity INT,
Salesperson_ID INT, Salesperson_Name VARCHAR(50)
);
INSERT INTO Sales_UNF VALUES
(1,101,'Rahul Sharma','Lucknow',201,'Laptop',2,301,'Amit Kumar'),
(2,102,'Priya Singh','Kanpur',202,'Mouse',3,302,'Neha Gupta'),
(3,103,'Arjun Verma','Delhi',203,'Keyboard',1,303,'Rohit Singh'),
(4,104,'Sneha Gupta','Lucknow',204,'Monitor',2,301,'Amit Kumar'),
(5,105,'Vivek Sharma','Kanpur',205,'Printer',1,302,'Neha Gupta');

SELECT * FROM Sales_UNF;

CREATE TABLE Sales_1NF (
Sale_ID INT,
Customer_ID INT,
Customer_Name VARCHAR(50),
Customer_City VARCHAR(50),
Product_ID INT,
Product_Name VARCHAR(50),
Quantity INT,
Salesperson_ID INT,
Salesperson_Name VARCHAR(50),
PRIMARY KEY (Sale_ID,Product_ID)
);
INSERT INTO Sales_1NF
SELECT * FROM Sales_UNF;

SELECT * FROM Sales_1NF;

CREATE TABLE Customer (
Customer_ID INT PRIMARY KEY,
Customer_Name VARCHAR(50),
Customer_City VARCHAR(50)
);

CREATE TABLE Product (
Product_ID INT PRIMARY KEY,
```

```
Product_Name VARCHAR(50)
);
CREATE TABLE Salesperson (
Salesperson_ID INT PRIMARY KEY,
Salesperson_Name VARCHAR(50)
);
CREATE TABLE Sales (
Sale_ID INT PRIMARY KEY,
Customer_ID INT,
Salesperson_ID INT,
FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
FOREIGN KEY (Salesperson_ID) REFERENCES Salesperson(Salesperson_ID)
);
CREATE TABLE Sale_Details (
Sale_ID INT,
Product_ID INT,
Quantity INT,
PRIMARY KEY (Sale_ID,Product_ID),
FOREIGN KEY (Sale_ID) REFERENCES Sales(Sale_ID),
FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
);
INSERT INTO Customer
SELECT DISTINCT Customer_ID,Customer_Name,Customer_City
FROM Sales_1NF;

INSERT INTO Product
SELECT DISTINCT Product_ID,Product_Name
FROM Sales_1NF;

INSERT INTO Salesperson
SELECT DISTINCT Salesperson_ID,Salesperson_Name
FROM Sales_1NF;

INSERT INTO Sales
SELECT DISTINCT Sale_ID,Customer_ID,Salesperson_ID
FROM Sales_1NF;

INSERT INTO Sale_Details
SELECT Sale_ID,Product_ID,Quantity
FROM Sales_1NF;


SELECT * FROM Customer;

SELECT * FROM Product;
```

```
SELECT * FROM Salesperson;

SELECT * FROM Sales;

SELECT * FROM Sale_Details;

SELECT
S.Sale_ID,
C.Customer_Name,
C.Customer_City,
P.Product_Name,
SD.Quantity,
SP.Salesperson_Name
FROM Sales S
JOIN Customer C ON S.Customer_ID=C.Customer_ID
JOIN Salesperson SP ON S.Salesperson_ID=SP.Salesperson_ID
JOIN Sale_Details SD ON S.Sale_ID=SD.Sale_ID
JOIN Product P ON SD.Product_ID=P.Product_ID;

SELECT
S.Sale_ID,
S.Customer_ID,
S.Salesperson_ID,
SD.Product_ID,
SD.Quantity
FROM Sales S
JOIN Sale_Details SD ON S.Sale_ID=SD.Sale_ID;

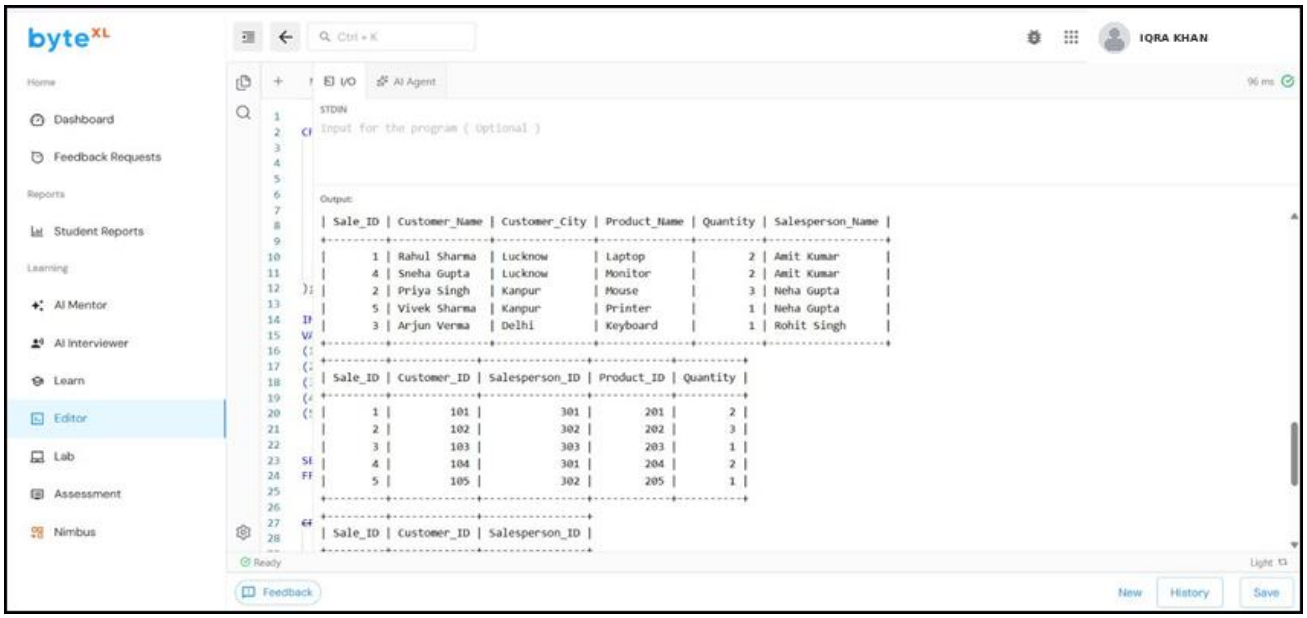
SELECT
Sale_ID,
Customer_ID,
Salesperson_ID

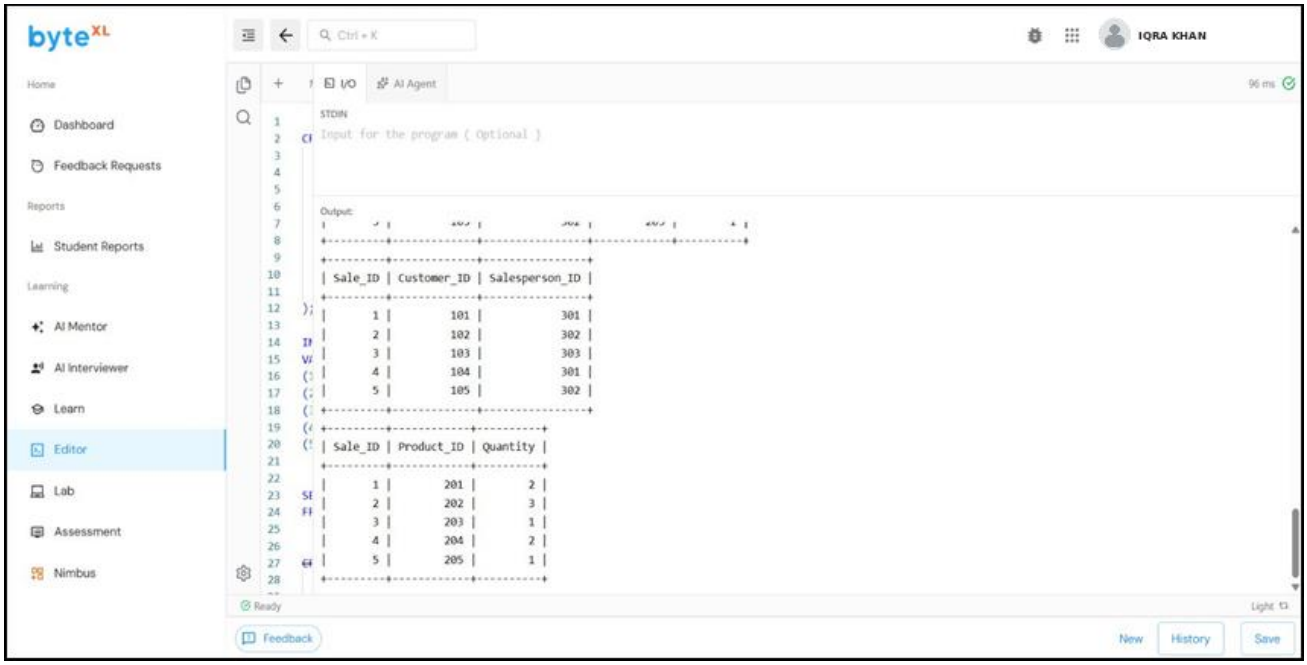
FROM Sales
WHERE Customer_ID IS NOT NULL;

SELECT
SD.Sale_ID,
SD.Product_ID,
SD.Quantity

FROM Sale_Details SD
    WHERE SD.Product_ID IS NOT NULL;
```

OUTPUT :


byteXL
IQRA KHAN
Dashboard
Feedback Requests
Student Reports
AI Mentor
AI Interviewer
Learn
Editor
Lab
Assessment
Nimbus
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Sale_ID	Customer_Name	Customer_City	Product_Name	Quantity	Salesperson_Name
1	Rahul Sharma	Lucknow	Laptop	2	Amit Kumar
4	Sneha Gupta	Lucknow	Monitor	2	Amit Kumar
2	Priya Singh	Kanpur	Mouse	3	Neha Gupta
5	Vivek Sharma	Kanpur	Printer	1	Neha Gupta
3	Arjun Verma	Delhi	Keyboard	1	Rohit Singh
Sale_ID	Customer_ID	Salesperson_ID	Product_ID	Quantity
1	101	301	201	2
2	102	302	202	3
3	103	303	203	1
4	104	301	204	2
5	105	302	205	1
Sale_ID	Customer_ID	Salesperson_ID
Ready
Light
Feedback
New
History
Save


byteXL
IQRA KHAN
Dashboard
Feedback Requests
Student Reports
AI Mentor
AI Interviewer
Learn
Editor
Lab
Assessment
Nimbus
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Sale_ID	Customer_ID	Salesperson_ID
1	101	301
2	102	302
3	103	303
4	104	301
5	105	302
Sale_ID	Product_ID	Quantity
1	201	2
2	202	3
3	203	1
4	204	2
5	205	1
Ready
Light
Feedback
New
History
Save